

Gaussian Process Regression

A Complete Mathematical Reference

1. What Is a Gaussian Process?

A Gaussian Process (GP) is a probability distribution over *functions* rather than over scalar values or vectors. Where a multivariate Gaussian distribution assigns probabilities to vectors of real numbers, a GP assigns probabilities to entire functions — infinite-dimensional objects — in a mathematically consistent and computationally tractable way.

The formal definition is: a collection of random variables $\{f(x) : x \in \mathcal{X}\}$ is a Gaussian Process if, for any finite set of input points $\{x_1, x_2, \dots, x_n\}$, the joint distribution of $(f(x_1), f(x_2), \dots, f(x_n))$ is multivariate Gaussian. A GP is completely specified by two functions:

$$f(x) \sim \mathcal{GP}(m(x), k(x, x'))$$

where $m(x) = \mathbb{E}[f(x)]$ is the **mean function**, encoding prior beliefs about the average behavior of f , and $k(x, x') = \text{Cov}(f(x), f(x'))$ is the **kernel function** (or covariance function), encoding beliefs about the smoothness and structure of f .

A helpful intuition: instead of saying “I believe the reward at intensity 0.7 is approximately 0.8”, a GP says “I maintain a probability distribution over all functions that could plausibly describe the reward landscape, and I update that distribution as I collect data.” This is a fundamentally richer representation of uncertainty.

2. The Kernel Function

The kernel is the most important design choice in a GP. It determines what kinds of functions are a priori plausible — how smooth they are, how quickly they vary, whether they exhibit periodicity, and so on.

A valid kernel must be **positive definite**: for any finite set of points $\{x_i\}$ and any vector $\mathbf{v}$, the kernel matrix K with entries $K_{ij} = k(x_i, x_j)$ must satisfy $\mathbf{v}^T K \mathbf{v} \geq 0$. This is not just a technical nicety — it ensures that the

resulting covariance matrix defines a genuine probability distribution.

2.1 The Squared Exponential (RBF) Kernel

$$k_{\text{SE}}(x, x') = \exp\left(-\frac{(x - x')^2}{2\ell^2}\right)$$

The length scale ℓ controls the characteristic distance over which the function varies. Functions drawn from a GP with this kernel are infinitely differentiable — unrealistically smooth for most physical processes. This kernel is mathematically convenient but can be a poor fit for real-world signals.

2.2 The Matérn Family

The Matérn kernel introduces a smoothness parameter ν that directly controls the differentiability of sample functions. Functions drawn from a Matérn- ν GP are exactly $\lceil \nu \rceil - 1$ times mean-square differentiable.

At half-integer values of ν , the Bessel functions in the general Matérn formula simplify to closed-form polynomials multiplied by exponentials. The three practically important cases are as follows.

Matérn 1/2 (equivalent to the Ornstein-Uhlenbeck process):

$$k_{1/2}(r) = \exp\left(-\frac{r}{\ell}\right), \quad r = |x - x'|$$

This produces continuous but nowhere-differentiable functions — like Brownian motion.

Matérn 3/2:

$$k_{3/2}(r) = \left(1 + \frac{\sqrt{3}r}{\ell}\right) \exp\left(-\frac{\sqrt{3}r}{\ell}\right)$$

This produces once-differentiable functions.

Matérn 5/2:

$$k_{5/2}(r) = \left(1 + \frac{\sqrt{5}r}{\ell} + \frac{5r^2}{3\ell^2}\right) \exp\left(-\frac{\sqrt{5}r}{\ell}\right)$$

This produces twice-differentiable functions — smooth enough to have well-defined gradients and curvature, but rough enough to model real-world signals that don't behave like glass. The Matérn 5/2 has become the standard workhorse kernel in machine learning for exactly this reason: it occupies the sweet spot between excessive smoothness and excessive roughness.

To understand the formula intuitively, separate it into two parts. The exponential term $\exp(-\sqrt{5}r/\ell)$ ensures similarity decays to zero as distance grows. The polynomial term $(1 + \sqrt{5}r/\ell + 5r^2/3\ell^2)$ modulates the rate of that decay, producing a

gentler falloff than a pure exponential. At zero distance, the kernel equals exactly 1 (perfect correlation). At $r = \ell$ it equals approximately 0.32 — inputs separated by one length scale share about 32% correlation.

2.3 The Periodic Kernel (Exp-Sine-Squared)

$$k_{\text{per}}(r) = \exp\left(-\frac{2\sin^2(\pi r / p)}{\ell^2}\right)$$

where p is the period and ℓ is a length scale controlling how rapidly correlation decays within each period. This kernel is appropriate when the function is expected to repeat on a known or learnable cycle.

2.4 Kernel Composition

New kernels can be created by combining existing ones. The sum $k_1 + k_2$ and product $k_1 \cdot k_2$ of valid kernels are both valid kernels. This allows composite kernels like the **spatio-temporal kernel**:

$$k_{\text{ST}}(x, x', t, t') = k_{\text{Matérn}}(x, x') \cdot k_{\text{per}}(t, t')$$

which models a function that is spatially smooth (in the x dimension) and temporally periodic (in the t dimension). The product structure means the function must be smooth *and* periodic simultaneously, which is appropriate when both local structure and global periodicity are expected.

3. GP Regression: The Posterior

Given n observations $\mathbf{y} = (y_1, \dots, y_n)$ at inputs $\mathbf{X} = (x_1, \dots, x_n)$, where each observation is a noisy evaluation of the true function: $y_i = f(x_i) + \epsilon_i$, $\epsilon_i \sim \mathcal{N}(0, \sigma^2)$, the GP posterior at a new set of test points $\mathbf{X}_*$ is a Gaussian distribution with mean and covariance given by:

$$\mu_* = K(\mathbf{X}_*, \mathbf{X}) \left[K(\mathbf{X}, \mathbf{X}) + \sigma^2 I \right]^{-1} \mathbf{y}$$

$$\Sigma_* = K(\mathbf{X}_*, \mathbf{X}_*) - K(\mathbf{X}_*, \mathbf{X}) \left[K(\mathbf{X}, \mathbf{X}) + \sigma^2 I \right]^{-1} K(\mathbf{X}, \mathbf{X}_*)$$

where $K(A, B)$ denotes the matrix with entries $k(a_i, b_j)$.

The posterior mean μ_* is the best estimate of the function at new points. The posterior covariance Σ_* quantifies uncertainty: it starts at the prior variance $k(x_*, x_*)$ and is reduced by the term $K_{\mathbf{X}}(K_{\mathbf{X}\mathbf{X}} + \sigma^2 I)^{-1} K_{\mathbf{X}}$, which represents the information gained from the training data. Where observations cluster densely, this reduction is large and uncertainty is small. Far from any observations, the posterior reverts to the prior.

4. Efficient Inference via Cholesky Decomposition

The naive approach to computing the posterior requires inverting the $n \times n$ matrix $(K_{XX} + \sigma_n^2 I)$, which costs $O(n^3)$ operations and is numerically unstable for near-singular matrices. The standard solution is the **Cholesky decomposition**.

For any symmetric positive definite matrix A , there exists a unique lower triangular matrix L with positive diagonal entries such that $A = LL^T$. The Cholesky decomposition costs $O(n^3/3)$ — the same asymptotic complexity as inversion, but with a smaller constant and, crucially, far better numerical stability.

The efficiency comes from the properties of triangular systems. To solve $Lz = b$, you can use **forward substitution**: compute $z_1 = b_1/L_{11}$, then $z_2 = (b_2 - L_{21}z_1)/L_{22}$, and so on — each variable follows directly from the ones computed before it, in $O(n^2)$ operations. Similarly, $L^Tz = b$ can be solved by **backward substitution**.

The full posterior computation then proceeds as follows. Let $L = \text{chol}(K_{XX} + \sigma_n^2 I)$.

For the posterior mean: $\alpha = L^{-T}(L^{-1}\mathbf{y})$, $\mu_* = K_{X*}\alpha$

For the posterior variance: $V = L^{-1}K_{X*}$, $\Sigma_* = K_{**} - V^TV$

The key insight in the variance computation is that V^TV equals $K_{X*}^TK_{XX} + \sigma_n^2 I)^{-1}K_{X*}$ without ever forming the explicit inverse — a significant savings in both memory and numerical error.

4.1 Positive Definiteness and the Jitter Term

The Cholesky decomposition only exists when the matrix is strictly positive definite. In practice, floating-point arithmetic can push tiny eigenvalues slightly negative, especially when observation points are close together and the kernel matrix entries are highly similar. Adding a small diagonal **jitter term** ϵI (typically $\epsilon \approx 10^{-5}$ to 10^{-3}) nudges all eigenvalues upward, guaranteeing the decomposition succeeds.

This jitter has a clean statistical interpretation: it is equivalent to assuming the observations have at least a small amount of Gaussian noise, which is almost always true in practice.

5. Hyperparameter Optimization via Marginal Likelihood

The kernel hyperparameters θ (length scale ℓ , noise variance σ_n^2 , etc.) are free parameters of the model. Rather than setting them by hand, they can be optimized by maximizing the **log marginal likelihood** — the probability of the observed data $\mathbf{y}$ integrated over all possible function values f :

$$\log p(\mathbf{y} \mid X, \theta) = -\frac{1}{2} \mathbf{y}^T K_{\theta}^{-1} \mathbf{y} - \frac{1}{2} \log |K_{\theta}| - \frac{n}{2} \log(2\pi)$$

where $K_{\theta} = K(X, X) + \sigma_n^2 I$ evaluated at hyperparameters θ .

The three terms have clean interpretations. The first term, $-\frac{1}{2} \mathbf{y}^T K_{\theta}^{-1} \mathbf{y}$, is a **data fit** term — it rewards hyperparameters that explain the observed data well. The second term, $-\frac{1}{2} \log |K_{\theta}|$, is a **complexity penalty** — it penalizes models that spread probability mass widely (large determinant means high entropy). These two terms trade off against each other naturally, producing an automatic Occam's razor: the marginal likelihood prefers the simplest model that fits the data.

Both terms are efficiently computable from the Cholesky factor L : $\mathbf{y}^T K_{\theta}^{-1} \mathbf{y} = |\alpha|^2 = \mathbf{y}^T \alpha$, $\log |K| = 2 \sum_i \log L_{ii}$

The gradient of the log marginal likelihood with respect to θ is also available in closed form, enabling gradient-based optimization (e.g., L-BFGS-B).

5.1 The Two-Timescale Structure

Hyperparameter optimization naturally operates on a slower timescale than posterior inference. Every new observation immediately updates the posterior (fast timescale: $O(n^2)$ for a rank-1 update). Hyperparameter re-optimization requires re-solving the full $n \times n$ system and running iterative optimization (slow timescale: $O(n^3)$ per objective evaluation, plus multiple evaluations).

This separation suggests a **logarithmic optimization schedule**: re-optimize at observation counts $n_0, 2n_0, 4n_0, 8n_0, \dots$ This front-loads optimization when the hyperparameter estimates are most uncertain (each observation reveals something new about global structure) and reduces computational burden as estimates stabilize.

The convergence behavior of this scheme is governed by the Robbins-Monro framework for stochastic approximation: if updates are made with step sizes $\alpha_t = c/t^{\rho}$ where $0.5 < \rho \leq 1$, then hyperparameter estimates converge to the true values under mild regularity conditions.

6. Computational Complexity and Sparse Approximations

Standard GP inference scales as $O(n^3)$ in time and $O(n^2)$ in memory, which becomes prohibitive for $n \gtrsim 10^4$. Several approximation strategies have been developed.

Sparse GP / Inducing Points (Nyström Approximation): Select $m \ll n$ inducing points Z that summarize the data. The full kernel matrix is approximated as $K_{XX} \approx K_{XZ}K_{ZZ}^{-1}K_{ZX}$, reducing complexity to $O(nm^2)$.

Random Fourier Features: By Bochner's theorem (see the Non-Stationary GP document), any shift-invariant kernel has a spectral representation. Approximating this with D random frequency samples converts kernel regression to linear regression in a D -dimensional feature space, costing $O(nD)$.

Kronecker Structure: When inputs lie on a grid and the kernel factorizes as a product over dimensions, the kernel matrix has Kronecker structure $K = K_1 \otimes K_2 \otimes \dots \otimes K_d$. Cholesky decomposition then costs $O(n^{3/d})$ instead of $O(n^3)$, an exponential improvement in dimension.

See also: Kernel Theory and the Matérn Family; Non-Stationary Gaussian Processes; Multi-Output GP and Coregionalization